From Research Code to CRAN: LLM-Assisted Refactoring of orthoMTL for Survival Endpoints

We Let Five AI Agents Prep a CRAN Package

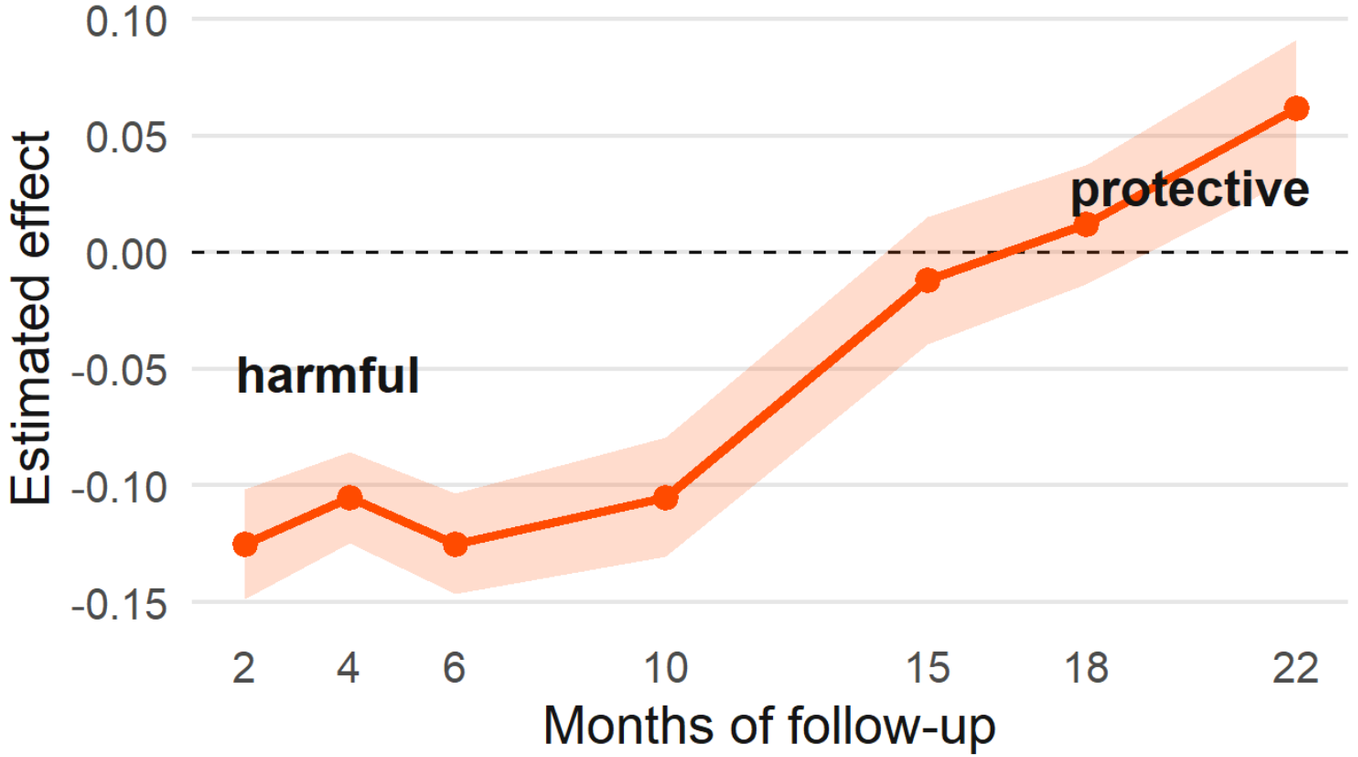
Kevin Vervier **Nadia Solovieff** **Gunther Jansen**

Novartis · R/Pharma 2026 · 20 October 2026

A biomarker that changes its mind

In a late-stage oncology trial, one baseline biomarker looked **unassociated with survival**.

A standard survival model scored its effect at ≈ 0.



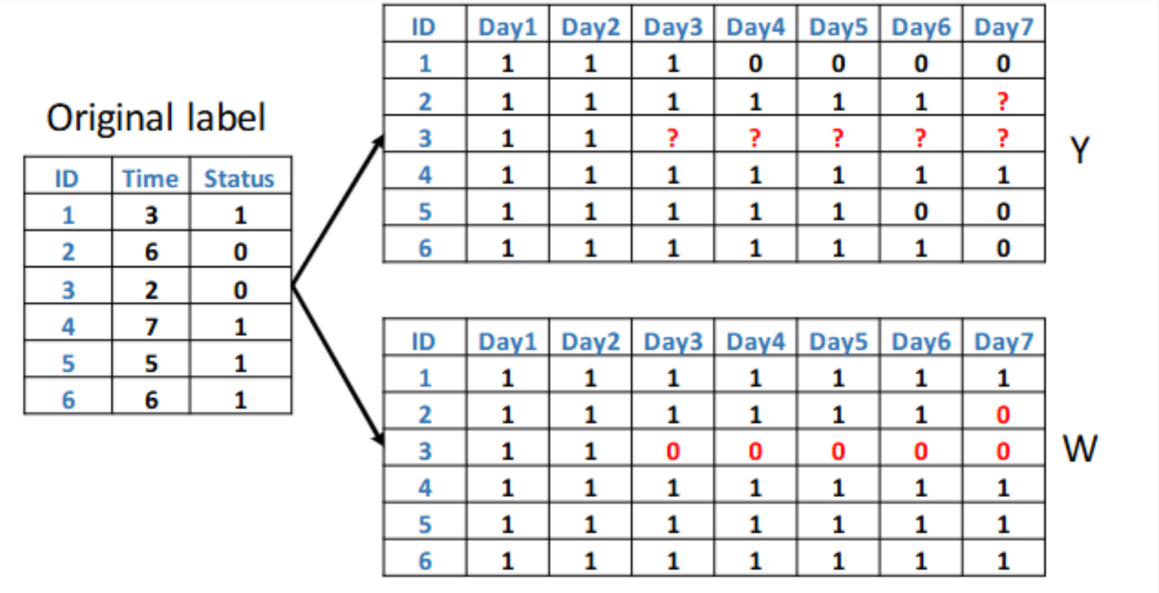
Schematic of one biomarker's effect. Shape is illustrative.

Harmful early. Protective late. The two halves cancelled out.

orthoMTL: don't average over time

Learn **several related tasks together**, sharing strength where they agree and staying apart where they don't.

Built first for **regression and classification**. Survival came later: cut follow-up into **landmarks**, fit them **at once**, each keeping its own effect.



Survival-as-multi-task framing after MTLSA (Li et al., KDD 2016).

`github.com/Novartis/orthoMTL` — on CRAN, 2026

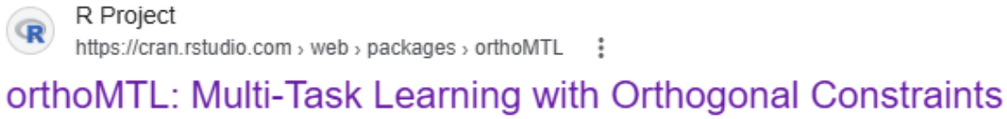
NOVARTIS

From PhD code to CRAN

Twelve-year-old research code. An agent did the packaging work.

Writing unit tests for the **logistic loss**, the agent found a **real bug** in my own 2014 gradient — labels were ± 1, the code used the $0/1$ formula. I checked it by hand: it was real, and the code was fixed.

The test it wrote: flipping every label **must** flip the answer. It didn't.



NOVARTIS

Then CRAN rejected us. Twice.

1. An invalid `LICENSE` field.
2. A hard-coded `set.seed()`.

Neither was caught by any automated check. Both were caught by **a CRAN volunteer**.

Would a skill file have got us there in one shot?

An agent did the refactoring. An agent did the packaging. Then **two rounds of human review**, on two things `R CMD check` never printed. The package is now on CRAN — so the question is the **counterfactual**.

Re-break the package with four defects — **no NAMESPACE**, a **LICENSE template**, **DESCRIPTION metadata**, **unused Imports** (three of the four are visible to `R CMD check`). Hand it to an agent under **five conditions**. Frozen prompt, fresh copy each time, results **blinded** before scoring.

The literature was already hedging: skills “often provide no benefit, and can even hurt” — SkillDelta (Xu et al. 2026), SkillAxe (Gautam et al. 2026). Skills from posit-dev/skills and

The result

0

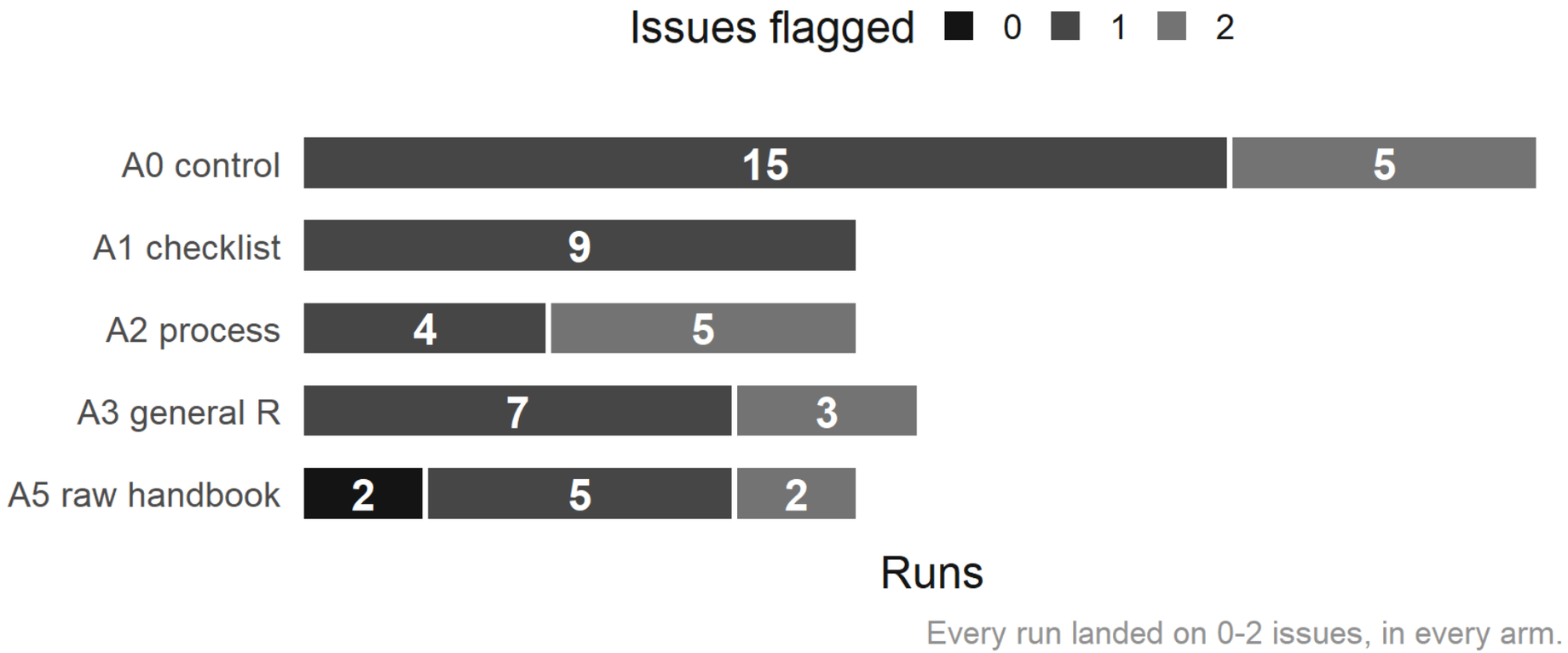
difference between the best skill file and **no skill file at all** — on the four seeded defects

All **57/57** runs fixed all four defects — including the control. None of the four skill files beat writing nothing at all.

That includes the LICENSE defect that actually got us rejected: **fixed in every run, in every arm**. Likely a ceiling: three of the four defects are ones `R CMD check` reports.

Then I went looking for why — and the interesting answer was in my own instrument.

Every arm lands in the same place



Every run landed on 0-2 issues, in every arm.

Every run, in every arm, ended with **0–2** reviewer-flagged issues. The control's spread is the same as everyone else's.

NOVARTIS

Why the control did so well

The agent runs `R CMD check` and reads its own failures.

A machine oracle beats a checklist: it reflects **this** package, **today**.

Skill files add little where a tool already tells the truth.

Not unique to us: SkillFlow found agents used retrieved skills **70%** of the time and still gained **nothing**. Reading a skill is not the same as needing one.

Not an argument against skill files — an argument about **where** they pay. The CRAN rules a skill encodes are the ones `R CMD check` already enforces.

Where the tool is silent — the hard-coded `set.seed()` — the arms **did** differ: the process skill and the raw handbook mostly fixed it, the control never did. Rules

The part I did not expect

`R CMD check` floored because it **works**. So the judgement oracle was supposed to see what the tool cannot.

It had **12 criteria**. Ten never fired once.

The two that did fire were the judge **disagreeing with itself** about text that **no arm had touched**:

- the maintainer email — unchanged in all **57** runs; read as fine in 10 trees, as a problem in 47
- one sentence in the seed — flagged in 23 trees, *excused* in 2, and in 29 the judge never opened the file (3 n/a)

Both “findings” were my oracle. Not my arms.

What I am *not* claiming

- **Not** “skill files are useless” → where tasks are *selected* to need them, they help a lot; this is *no benefit on CRAN prep, with the docs at hand*
- **N = 1** on every axis that matters: one package, one model (Claude Opus 5.5 in Claude Code), five instruction sets — and runs were not powered, just as many as time and budget allowed
- **No positive control in the blinded run**: I planned a hindsight arm, but its file was never delivered — I found out afterwards. A later run with the file delivered did fix the seed defect, so the instrument can see an effect there; but the judge panel (previous slide) says “no sensitivity” is still live

Postscript: our own maintainer address

Even a noisy judge can be right once. It flagged our own maintainer address in 47 of 57 trees:

`kevin.vervier@novartis.com`

CRAN requires **a person, not a mailing list**, kept current and reachable “**without any form of filtering**”. A corporate account is one re-org away from failing all three.

Not a violation today. Changing in 0.2.0 anyway.

Take home

1. Learning related tasks **jointly or separately** is the general tool; survival is its most recent use case → `orthoMTL`

2. Agents are good at checks a **machine** can run. Invest in **executable oracles** — tests, `R CMD check`, symmetry checks — before prose guidance.

3. I set out to measure skill files. I couldn't — my judge measured itself. Finding out **why** was worth more than the answer.

Generation got cheap. Verification did not.

`github.com/Novartis/orthoMTL` · Vervier et al. 2014 · Juric et al. 2026 · full references in backup

Backup

NOVARTIS

Backup: what the five arms were

Arm	Skill file	Source	Runs
A0	none (control)	—	**20** (10 + 10, see below)
A1	`cran-extrachecks`	posit-dev/skills	9
A2	`cran-submit`	CoryMcCartan/agent-skills	9
A3	`r-package-development`	posit-dev/skills	10
A5	raw handbook text	CRAN docs, next slide	9

Every arm — **A0 included** — also had the same offline CRAN docs on disk. 57 of 60 planned runs completed; 3 ended on API rate limits.

Agent: Claude Opus 5.5 (`claude-opus-5-5`) in Claude Code 2.1.282–284 (auto-updated mid-experiment). Judge: Claude Sonnet 5 — same vendor family as the agent. Skill files were delivered as an always-loaded `CLAUDE.md`, not through the on-demand Skill tool, which the agent did not have. A0 has 20 runs because a second batch of 10 (labelled A4) was meant to carry a hindsight file written from the rejection history and never received it: first-call context was 7,145 tokens in all 10, identical to A0. They are pooled as A0. A later paired run with the file delivered is on the last backup.

Backup: the handbook every arm could read

Snapshot captured **2026-09-18**, identical across all 60 runs.

Source	URL
CRAN Cookbook	contributor.r-project.org/cran-cookbook
CRAN Repository Policy	cran.r-project.org/web/packages/policies.html
CRAN Submission Checklist	cran.r-project.org/web/packages/submission_checklist.html

The Cookbook is built from real CRAN re-submissions and co-authored by a member of the CRAN team — the closest public thing to the reviewers' own knowledge. A5 loaded this text directly into context; the other arms were told where it was.

Backup: the hedges, in their own words

Finding	Source
Skills “often provide no benefit, and can even hurt performance while incurring additional token costs”	SkillDelta, Xu et al. 2026
Human-authored skills +16.2 pp; **LLM-authored skills give no measurable gain**	SkillAxe, Gautam et al. 2026
Agents used retrieved skills 70% of the time yet showed **no performance gain** where the corpus lacked good skills for the domain	SkillFlow, Li et al. 2026
Self-generated skills landed **below** the no-skills baseline (−8 to −11.5 pp)	SkillsBench, Li et al. 2026

My null is the same shape, in a domain none of them covered.

Backup: the counterweight

The headline positive result is **SkillsBench**: curated skills lift pass rate 33.9% → 50.5% (**+16.6 pp**) across 87 tasks and 18 model–harness configurations. I am not disputing it. Three things make it a different question from mine:

- **8 domains** — natural science, media, cybersecurity, industrial, finance, office, software engineering, maths/OR. **No R, no packaging, no CRAN**
- Tasks are admitted only if **“significantly easier with Skills”**; no-separation tasks are rejected as low-signal
- Skills are drawn from the **top quality quartile** (10.1/12 vs ecosystem mean 6.2/12); the authors call this “an optimistic scenario”

Their own domain gradient points my way: **software engineering +11.6 pp**, second-lowest of eight, against natural science **+28.8 pp** — “domains with stronger pretraining and tooling coverage benefit less.”

Backup: full references

Package and method

- orthoMTL — https://github.com/Novartis/orthoMTL
- Predecessor `orthopen` — https://github.com/kevinVervier/orthopen
- Vervier K., Mahé P., Vert J.-P. (2014). *Benchmark of structured machine learning methods for microbial identification from mass-spectrometry data*. Springer. — method origin
- Li Y. et al. (2016). *A Multi-Task Learning Formulation for Survival Analysis*. KDD. — survival-as-multi-task framing (MTLSA), source of the slide-4 diagram
- Juric D. et al. (2026). *Annals of Oncology*. — clinical application

Skill files under test

- Posit `skills` — https://github.com/posit-dev/skills
- McCartan C., `agent-skills` — https://github.com/CoryMcCartan/agent-skills

Prior measurement of agent skills

NOVARTIS

- Xu A. et al. (2026). *When Does a Skill Add Value? Task-Conditional Gain Prediction for Selective Skill Use*. arXiv:2609.32274
- Gautam S., Radhakrishna A., Gulwani S. (2026). *SkillAxe: Sharpening LLM-Authored Agent Skills Through Evaluation-Guided Self-Refinement*. arXiv:2606.10546
- Li F., Tagkopoulos P., Tagkopoulos I. (2026). *SkillFlow: Scalable and Efficient Agent Skill Retrieval System*. COLM 2026, arXiv:2504.06188
- Li X. et al. (2026). *SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks*. arXiv:2602.12670

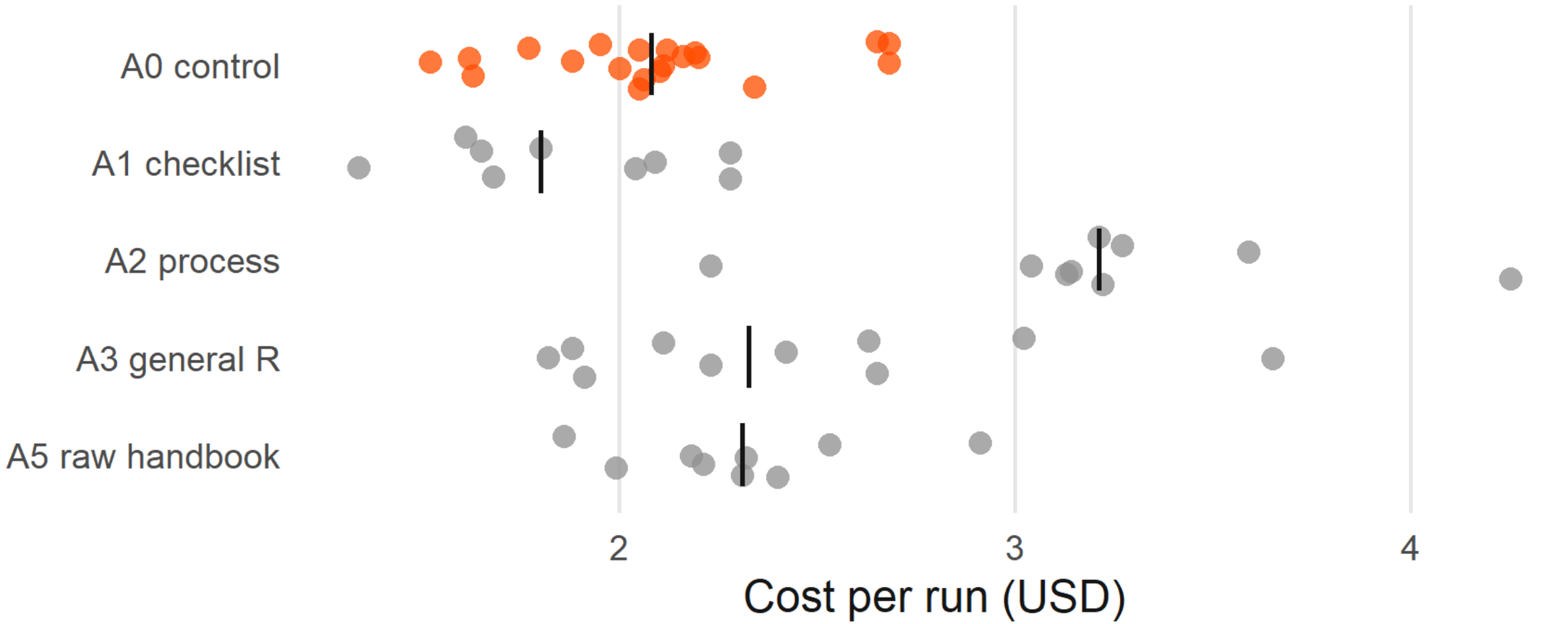
CRAN documentation

- R Core / CRAN team, *CRAN Cookbook* — https://contributor.r-project.org/cran-cookbook/
- *CRAN Repository Policy* — https://cran.r-project.org/web/packages/policies.html
- *CRAN Submission Checklist* — https://cran.r-project.org/web/packages/submission_checklist.html

Backup: the three ways we checked correctness

Check	What it proves
Against the old validated package	identical coefficients on 5 fixed cases
Flip every label	answer must flip; `0.62` → `0.000`
Simulated data, known truth	recovers the right features, 10/10 seeds

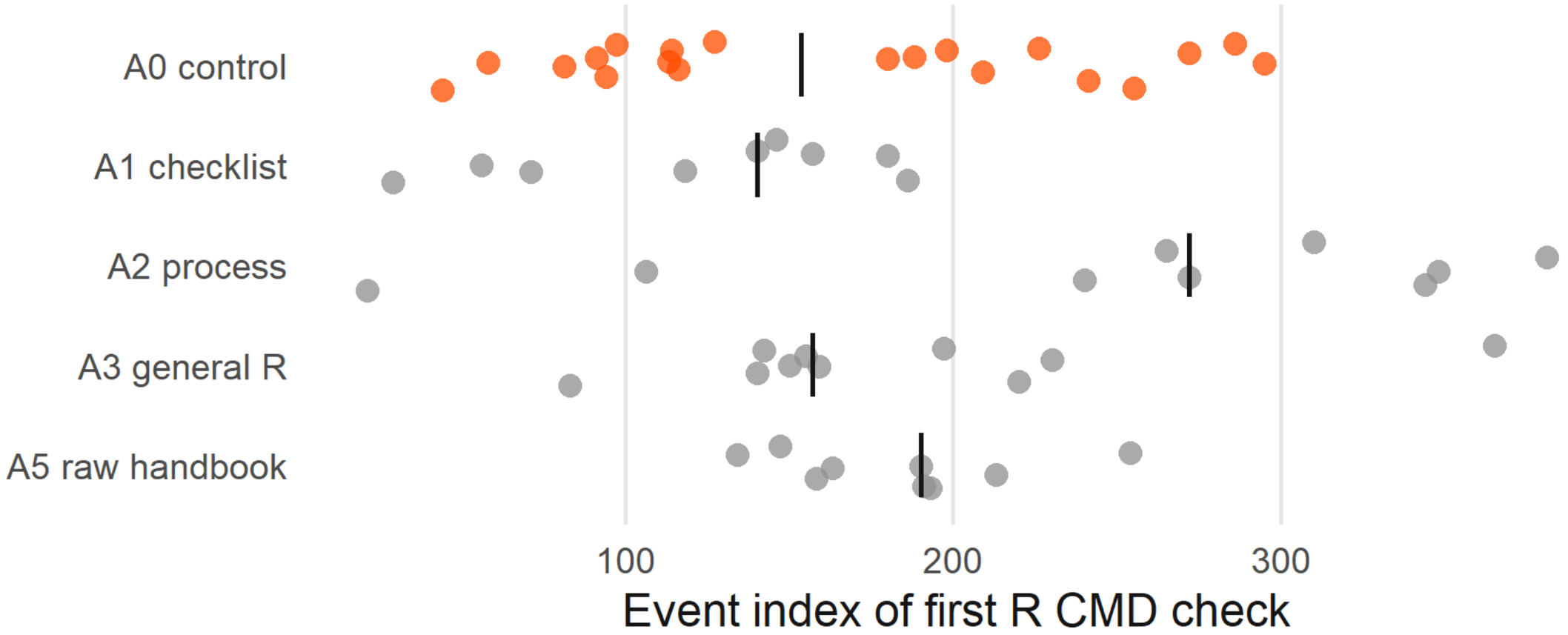
Backup: cost per run



Dots = runs; bar = median.

NOVARTIS

Backup: when did the agent first run `R CMD check`?



Dots = runs; bar = median.

NOVARTIS

Backup: what I would change

- Score **R01, the `set.seed()` finding**, with a rule that also sees seeds passed as default arguments, and add judge-side defects so the judge can discriminate
- Seed more defects only a **CRAN volunteer** could catch
- Add a **positive control**: an arm with the CRAN docs removed, so the skill file is the only guidance. If that arm does worse, the measurement has teeth
- Judge each package several times, not once
- More packages, more models

Backup: the four seeded defects

Defect	Caught by `R CMD check`?	Repaired
`NAMESPACE` deleted	yes — check cannot even run	57/57
`usethis` LICENSE template vs GPL-3 DESCRIPTION	**no** — this is rejection **R02**	**57/57**
DESCRIPTION metadata (`Authors@R`, title case)	yes	57/57
Unused declared `Imports`	yes	57/57

Three of four are machine-visible. Our other real rejection — **R01, the hard-coded `set.seed()`** — is a fifth defect that `R CMD check` does not report; it is on the next slide.

Three of four machine-visible is the result's main caveat: this measures the easy half of CRAN preparation.

Backup: the hard-coded seed (R01)

The starting package defaulted `seed = 42` and called `set.seed()` unconditionally in four exported functions — the exact pattern CRAN rejected. `R CMD check` does not report it.

Arm	n	Still hard-coded	Default kept, RNG state restored	Canonical fix (`seed = NULL`)
A0 control	20	10	10	0
A1 checklist	9	1	8	0
A2 process	9	1	1	7
A3 general R	10	3	7	0
A5 raw handbook	9	1	0	8

Completed runs only; descriptive counts, no test. “Default kept” means `seed = 42` stays but the value goes through a helper that restores the user’s RNG state — arguably acceptable, not what the reviewer asked for. A0 includes the 10 runs that were meant to carry the hindsight file and did not (see arms slide).

These counts are a re-score, run after unblinding. My original rule only matched `set.seed(<literal>)` inside a function body, so it scored **zero hits on the seed itself** — a detector that cannot see the defect it was built for. I found that while preparing this talk, fixed the rule, and re-ran it. The re-score reads package files only and never touches the blind map: the same code at first unblinding would have produced this same table. The flaw was in the detector, not in the direction of the correction.

Backup: positive control, run afterwards (R01)

The hindsight file was written from CRAN's own two rejection messages, installed, and checked to reach the model (first-call context +1,055 tokens over its paired control). Then A0 and the hindsight arm were run as 10 interleaved pairs, **after unblinding, labels known**, on a newer Claude Code (2.1.294).

Arm	n	Canonical fix (`seed = NULL`)	Default kept	Still hard-coded
Fresh A0	10	0	6	4
Hindsight file delivered	10	10	0	0

Two extra hindsight runs without a surviving partner (rate limits) were also canonical: 12 of 12. This is a ceiling check, not a skill-file result: the file contains the answer. It shows the seed defect is learnable from text in context, which the blinded experiment could not show. Not pooled with the 57; no judge panel was run on these trees. `R CMD check` was identical across arms: no errors or warnings in any of the 22 trees. Scripts and scores: `supplementary/`.

NOVARTIS